

A history-based cost-cognizant test case prioritization technique in regression testing

Yu-Chi Huang^c, Kuan-Li Peng^a, Chin-Yu Huang^{a,b,*}

^a Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

^b Institute of Information Systems and Applications, National Tsing Hua University, Hsinchu, Taiwan

^c ASUSTeK Computer Inc., Taipei, Taiwan

ARTICLE INFO

Article history:

Received 10 April 2011

Received in revised form 30 July 2011

Accepted 14 September 2011

Available online 17 November 2011

Keywords:

Software testing

Regression testing

Test case prioritization

Fault severity

Software development life cycle

ABSTRACT

Software testing is typically used to verify whether the developed software product meets its requirements. From the result of software testing, developers can make an assessment about the quality or the acceptability of developed software. It is noted that during testing, the test case is a pair of input and expected output, and a number of test cases will be executed either sequentially or randomly. The techniques of test case prioritization usually schedule test cases for regression testing in an order that attempts to increase the effectiveness. However, the cost of test cases and the severity of faults are usually varied. In this paper, **we propose a method of cost-cognizant test case prioritization based on the use of historical records. We gather the historical records from the latest regression testing and then propose a genetic algorithm to determine the most effective order.** Some controlled experiments are performed to evaluate the effectiveness of our proposed method. Evaluation results indicate that our proposed method has improved the fault detection effectiveness. It can also be found that prioritizing test cases based on their historical information can provide high test effectiveness during testing.

© 2011 Elsevier Inc. All rights reserved.

1. Introduction

In modern software development, the goal of software testing is to verify whether or not the developed software meets the requirements, such as performance, security, safety, correctness, and reliability (Lyu, 1996; Ammann and Offutt, 2008; Huang, 2005; Sharma and Trivedi, 2007). For instance, an unhandled invalid input often means a security vulnerability. This is why the classic stack-based buffer overflow attack can be made to work. A well-designed software system should be robust enough to behave reasonably with invalid inputs in the presence of unforeseen environmental or attack problems. To support security (and safety) as well as to evaluate the behavior and performance of developed software, it is useful to produce test cases that contain invalid inputs (Ammann and Offutt, 2008). It is also suggested that some best practices and standards like SSE-CMM, ISO/IEC 15408 and ISO/IEC 27002 be taken into consideration during software development to ensure basic security and safety requirements (Nunes et al., 2010).

As a software product is modified in order to fix detected/reported bugs or to add extra needed functions, new faults may be introduced into the software product. This

phenomenon is typically known as imperfect debugging (Lin and Huang, 2009a,b). In reality, developers experience cases where they fix one bug but could create another new one. Ohba and Chou (1989), for example, reported that about 14% of the faults detected and removed during an observation period introduced new faults as a result of imperfect debugging. Goel and Okumoto (1978) also showed that the imperfect debugging model provided a good fit to the software failure data from a real-time control system for a land-based radar system developed by Raytheon Company.

During software development life cycle (SDLC), regression testing can be conducted at each of the test levels, and it can be effectively used to reduce the possibility of imperfect debugging. Software engineers often reserve the test suites that they have developed, and reuse those test suites in the next regression testing. The simplest approach to perform regression testing is to reuse all of the test cases in the test suites. However, it may be very expensive and impracticable because this would need lots of test case executions. Hence, some existing approaches, such as the regression test selection, the test suite reduction, and the test case prioritization have been proposed and used to reduce the cost of regression testing (Kim et al., 2002).

For example, regression test selection approaches usually select a subset of the existing test suite with the most effectiveness, and rerun those selected test cases (Binkley, 1997; Engstrom et al., 2010; Leung and White, 1990; Rothermel and Harrold, 1997; Graves et al., 2001). But the test suite reduction approaches always

* Corresponding author at: Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan. Tel.: +886 3 5742972; fax: +886 3 5723694.
E-mail address: cyhuang@cs.nthu.edu.tw (C.-Y. Huang).

plan to reduce the test suite to a subset that has the same ability as the original test suite (Zhong et al., 2008; Harrold et al., 1993; Chen and Lau, 1998; Lin and Huang, 2009a,b). It can be seen that these approaches, however, have some weaknesses: fault detection capabilities of the test suite may decrease because these approaches reduce the cost by discarding some redundant test cases.

In general, test case prioritization approaches schedule test cases in an order so that test cases with a higher priority will be executed earlier (Rothermel et al., 2001; Elbaum et al., 2002, 2004; Li et al., 2007; Mei et al., 2009). Professional testers often order the test cases to meet some performance goal because their goal is to increase the fault detection rate of test suites within the regression testing process. Many existing techniques have been proposed to prioritize test cases for regression testing (Rothermel et al., 2001; Elbaum et al., 2002; Walcott et al., 2006). These techniques analyze source codes to obtain useful information such as the function coverage of test cases, and then prioritize these test cases. However, it is observed that these techniques have two limitations. First, these techniques have usually assumed (or considered) that test case costs and fault severities are uniform. Second, if testers cannot obtain the source code, applying these techniques will become very difficult.

In order to overcome above mentioned limitations and difficulties, cost-cognizant test case prioritization approaches and history-based test prioritization approaches were proposed and studied (Malishevsky et al., 2006; Kim and Porter, 2002; Park et al., 2008). In general, cost-cognizant test case prioritization approaches consider the varying test case costs and fault severities when prioritizing test cases (Malishevsky et al., 2006). Additionally, history-based test prioritization approaches prioritize test cases based on historical execution data (Kim and Porter, 2002). It is noted that some researches also report that genetic algorithm (GA) can be effectively applied in test case prioritization (Rothermel et al., 2001; Walcott et al., 2006).

In this paper, based on the records of historical test case execution, we will propose a *Modified Cost-Cognizant Test Case Prioritization* (MCCTCP) that can be used for software (security) regression testing. We will gather the test costs, the fault severities, and the detected faults of each test case from the latest regression testing, and then adopt GAs to find out an order with the greatest rate of “units of fault severity detected per unit test cost”. Numerical results based on some experiments will show that our proposed method can effectively improve the effectiveness of cost-cognizant test case prioritization without analyzing the source code, no matter whether test case costs and fault severities are uniform.

The rest of the paper is organized as follows: Section 2 reviews some related works. Next, our proposed approach is presented and discussed in Section 3. Some experiments will be performed and numerical results are analyzed in Section 4. Section 5 discusses the impact of generations and population size on the proposed method. Finally, Section 6 draws some conclusions.

2. Test case prioritization

Test case prioritization approaches typically sort existing test cases for regression testing according to performance goals. There are numerous diversity methods proposed in the literature (Rothermel et al., 2001; Elbaum et al., 2002). For example, coverage-based prioritization techniques prioritize test cases according to the quantity of statements or functions executed by the test cases. The test cases that execute more statements or functions will be applied to the regression testing earlier. In addition, fault exposing potential prioritization techniques typically schedule test cases based on the capability of the test case in terms of exposing the faults. These techniques incorporate the probability of the statements containing faults and the probability of the faults causing failures into

prioritization. If the test cases execute more faulty statements or functions that cause failures, they will be given higher priority. History-based test prioritization techniques sort the test cases in terms of the historical value calculated from the test case execution history, such as covered program entities (Kim and Porter, 2002). These techniques give higher priority to test cases that cover more program entities in previous regression testing, like functions and statements.

In order to generate a satisfactory testing order, many cost-cognizant test case prioritization approaches were proposed, and these approaches schedule test cases according to not only the coverage information, but also the varying test case costs and fault severities of each test case (Malishevsky et al., 2006; Park et al., 2008). Park et al. (2008) proposed a historical value model, which is used to quantify the historical value of a test case based on the historical test case costs and the historical fault severities for cost-cognizant test case prioritization. In addition, Jeffrey and Gupta (2006) proposed a method based on total statement coverage and the relevant slice. The relevant slice represents a set of statements that may influence the output of a program when a particular test case is executed. This approach gives higher priority to the test case that has more statements in its relevant slice.

Qu et al. (2007) proposed an algorithm based on test history and run-time information to prioritize test cases in a black-box environment. The algorithm groups all reused test cases according to the fault types they revealed, and then used this information to prioritize test cases dynamically when doing regression testing. Zengkai and Jianjun (2008) also presented an approach based on the analysis of program structure. They scheduled test cases according to the testing-importance of a module composed of fault proneness, and the importance of that module.

Because customer satisfaction can reflect the quality of software (Karlsson and Ryan, 1997), value-based approaches were proposed to measure user-perceived software quality (Boehm and Huang, 2003). In order to improve the quality, Srikanth et al. (2005) presented a system-level test case prioritization technique. They extended code-coverage test case prioritization techniques and scheduled test cases according to requirement volatility, customer priority, implementation complexity, and fault proneness of requirements. Sherriff et al. (2007) proposed a singular value decomposition-based regression test prioritization technique to order test cases. This technique uses matrixes to gather change records, and analyzes them using singular value decomposition to guide test prioritization. Since the Java and JUnit test frameworks are currently very popular, Do et al. (2004) designed and performed a controlled experiment to analyze the effectiveness of test case prioritization techniques on Java systems tested under the JUnit framework. They have shown that prioritization techniques also improve the effectiveness of the JUnit test suite and reveal differences from previous studies.

Finally, from above survey, it is worthy to note that in order to evaluate the effectiveness of the test case prioritization techniques, the *Average Percentage of Fault Detected* (APFD) metric has been widely used (Rothermel et al., 2001; Elbaum et al., 2002). APFD is a measure of the rate of fault detection of a test suite and its value ranges from 0 to 1. Higher APFD values indicate that the prioritized test suite has a greater rate of fault detection. It should be noted that the definition of APFD is based on the assumption that all test case costs are uniform and all fault severities are uniform. In fact, test case costs and fault severities are diverse, and APFD may generate inappropriate results in such situations. To overcome this limitation, another cost-cognizant metric, the *Average Percentage of Faults Detected per Cost* (APFDc) has been proposed (Malishevsky et al., 2006; Elbaum et al., 2001). This metric incorporates varying test case costs and fault severities into APFD. In other words, the metric, APFDc, is created by adapting the APFD metric to measure

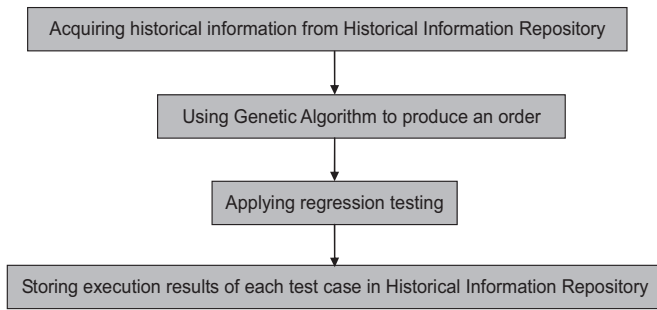


Fig. 1. Procedure of proposed MCCTCP method.

the rate of “units of fault severity detected per unit test cost” of a test order. If all fault severities and test case costs are uniform, the formula for APFDc will reduce to the formula for APFD.

3. Proposed MCCTCP method

3.1. Problem settings

Definition. The history-based test case prioritization problem: given: T , a test suite; $P(T!)$, the set of all permutations of T ; HR , the historical records of all elements in $P(T!)$; f , a function from $P(T!)$ to real numbers, and fit , a function from HR to real numbers.

Problem:

Find $T_{\max} \in P(T!)$ such that $\forall T' \in P(T'), (T' \neq T_{\max}),$

$$[fit(HR_{T_{\max}}) \geq fit(HR_{T'})] \text{ and } [f(T_{\max}) \geq f(T')] \quad (1)$$

In Eq. (1), function fit evaluates the award value of the order based on its historical record, and function f quantifies the effectiveness of the order. The goal of this problem is to find the order that demonstrates the goodness in the past and the effectiveness in future regression testing. That is, we would like to search for an order of the existing test suite which has the greatest effectiveness in terms of cost-cognizant test case prioritization in regression testing. Since this search problem, which is a kind of NP-hard problem, can be solved by *genetic algorithm* (GA) (Li et al., 2007), we input the historical execution information of the test cases to a GA to produce such an order. Fig. 1 briefly depicts the procedure of our proposed MCCTCP method.

As shown in Fig. 1, the *historical information repository* is used to store the historical information of each test case in regression testing, including the cost of the test case, the faults detected by the test case, and the fault severity of detected faults. When prioritization is applied, the proposed MCCTCP method first acquires the stored historical information of each test case and inputs this data to GA to produce an order that has the greatest effectiveness in terms of the preceding regression testing. After applying regression testing, the execution results of each test case will be stored in the historical information repository. Since GA uses a heuristic search considering the costs of test case and the severities of faults to find the optimal test order, this order would in reality be more appropriate for regression testing. Note that MCCTCP method utilizes the historical information to schedule the test cases, thus it does not need to analyze the source code of the tested program.

3.2. Proposed genetic algorithm

Our proposed GA is shown in Fig. 2 and it is used to schedule the test cases in test suite T based on their historical information, such as the cost, the detected faults, and the fault severities. As we can see on line 1 in Fig. 2, GA algorithm acquires the list of test cost,

Algorithm TestCasePrioritization

Input:

T : Test suite
 p : Population size
 g : Number of generation
 cp : Crossover probability
 mp : Mutation probability

Output:

T_{greatest} : A test order which has greatest fitness value in population in final generation

1. Acquire test cost list, tc , detected fault list, fl , and fault severity list, fsl , from Historical Information Repository
2. $P_1 \leftarrow \text{genPopulation}(T, p, fl, fsl)$
3. **for** $i = 1$ **to** g
4. $F_i \leftarrow \text{evaFitness}(P_i, tc, fl, fsl)$
5. $P_{i+1} \leftarrow \text{addTwoBest}(F_i, P_i)$
6. **for** $j = 3$ **to** p
7. $\text{parent}_1 \leftarrow \text{randSelectParent}(P_i)$
8. $\text{parent}_2 \leftarrow \text{randSelectParent}(P_i)$
9. $\text{child}_1, \text{child}_2 \leftarrow \text{crossover}(\text{parent}_1, \text{parent}_2, cp)$
10. $\text{child}_1 \leftarrow \text{mutation}(\text{child}_1, mp)$
11. $\text{child}_2 \leftarrow \text{mutation}(\text{child}_2, mp)$
12. $P_{i+1} \leftarrow \text{addChildren}(\text{child}_1, \text{child}_2)$
13. $F_{g+1} \leftarrow \text{evaFitness}(P_{g+1}, tc, fl, fsl)$
14. $T_{\text{greatest}} \leftarrow \text{selectBestChild}(F_{g+1}, P_{g+1})$
15. **return** T_{greatest}

Fig. 2. The proposed Genetic Algorithm to prioritize test case.

tc , the list of detected fault, fl , and the list of fault severity, fsl , from the historical information repository. On the lists of test cost and detected fault is recorded the cost of each test case and the detected faults of each test case in the preceding regression testing respectively. The list of fault severity records the severity of each fault in the preceding regression testing. In addition, on line 2 in Fig. 2, the first population which contains p chromosomes is generated by $\text{genPopulation}(T, p, fl, fsl)$. The encoding of chromosomes is permutation encoding. In permutation encoding, each chromosome is a sequence of test case numbers that represents the execution order of test cases. For example, a test suite of five test cases and the exposed faults of each test case are shown in Table 1, and two chromosomes are shown in Fig. 3. The chromosome A and chromosome B represent the test case order $t_5-t_3-t_1-t_4-t_2$ and $t_3-t_2-t_5-t_4-t_1$ respectively.

Furthermore, Fig. 4 gives the algorithm of $\text{genPopulation}(T, p, fl, fsl)$. First, this algorithm calculates the sum of detected fault severities for each test case and sorts the test cases in decreasing order according to that sum. This decreasing order is added to the first population because we would like to let the test order find the critical faults with higher severities as soon as possible, and to ensure that the effectiveness of the final order produced by the

Table 1
Example of test cases and exposed faults.

Test cases	Faults				
	1	2	3	4	5
t_1	X			X	
t_2		X			X
t_3	X	X	X		
t_4		X			X
t_5	X		X		X

Chromosome A:

5	3	1	4	2
---	---	---	---	---

Chromosome B:

3	2	5	4	1
---	---	---	---	---

Fig. 3. Example of chromosomes.

TestCasePrioritization algorithm is at least better than this order. Finally, *genPopulation* generates $p - 1$ test case orders and adds them to the first population.

Note that in the proposed genetic algorithm in Fig. 2, in the for loop beginning on line 3, the evolution process is performed. The algorithm evaluates the fitness value of each chromosome and stores them in F_i on line 4. On line 5, the algorithm adds two chromosomes that have the best fitness value to the new population and stores them in P_{i+1} . These chromosomes are added to the new population to ensure that at least two best chromosomes survive, and the best chromosome in the new population is at least better than these chromosomes.

In the inner loop beginning on line 6 in Fig. 3, the reproduction process is performed for the evolution. The algorithm selects two parents on line 7 and line 8 by *randSelectParent*(P_i). The algorithm of *randSelectParent*(P_i) is shown in Fig. 5. The parent is selected according to roulette wheel selection method. If the chromosome has a greater fitness value, it has more chances to be selected. The *randSelectParent* first calculates the *fitSum*, which is the sum of fitness values of all chromosomes in the population, and then generates a random number r that ranges from 0 to *fitSum*. The *randSelectParent* next goes through the population and subtracts *fitOfChromosome_j*, which is the fitness value of the chromosome j , from r . The first chromosome that makes $r < 0$ is selected.

As shown in Fig. 2, after selecting two parents, the *crossover*(*parent₁*, *parent₂*, *cp*) may recombine the selected parents based on the crossover probability, *cp*, on line 9. The *crossover*(*parent₁*, *parent₂*, *cp*) produces two new chromosomes, *child₁* and *child₂*. On lines 10 and 11, the *mutation*(*child*, *mp*) mutate the child based on the mutation probability. On line 12, the algorithm adds two new chromosomes generated by the reproduction process to the new population. Additionally, as seen from Fig. 2, the reproduction process on lines 6–12 is repeated until the new population has p chromosomes. The evolution process on lines 3–12 is applied g times to evolve the population. When the final population,

Algorithm genPopulation**Input:**

T : Test suite
 p : Population size
 fl : Detected fault list
 fsl : Fault severity list

Output:

P_1 : The first population

1. $sumList \leftarrow sumFaultSeverity(T, fl, fsl)$
2. $chromosome_{sort} \leftarrow sortSum(T, sumList)$
3. $P_1 \leftarrow addChromosome(chromosome_{sort})$
4. **for** $i = 1$ **to** p
5. $chromosome \leftarrow randGenerateSequence(T)$
6. $P_1 \leftarrow addChromosome(chromosome)$
7. **return** P_1

Fig. 4. The genPopulation algorithm.**Algorithm randSelectParent****Input:**

P_i : The population of i -th generation

Output:

parent: The chromosome selected by the algorithm

1. $fitSum \leftarrow calFitSumOfChromosomes(P_i)$
2. $r \leftarrow genRandNum(fitSum)$
3. **for** $j = 1$ **to** p
4. $r = r - fitOfChromosome_j$
5. **if** $r < 0$
6. **break**
7. $parent \leftarrow chromosome_j$
8. **return** *parent*

Fig. 5. The randSelectParent algorithm.**Table 2**

Example of test cases and test costs.

Test case	Test cost
t1	4
t2	2
t3	1
t4	3
t5	2

P_{g+1} , is produced, the fitness value of each chromosome is determined on line 13, and the chromosome whose fitness value is the greatest is selected to be the test order.

3.2.1. Fitness function

Referring to Fig. 2, the *evaFitness*(P_i , *tc*, *fl*, *fsl*) on lines 4 and 13 evaluates a fitness value of each chromosome in the population through the fitness function. Since the goal of our proposed method is to search out an order of the existing test suite that has the greatest effectiveness in terms of cost-cognizant test case prioritization in the preceding regression testing, *evaFitness*(P_i , *tc*, *fl*, *fsl*) utilizes historical information to calculate the fitness value according to the cost-cognizant metric, APFDc. In other words, the fitness value is the APFDc value of the chromosome in the preceding regression testing. If there are m faults and n test cases for a tested program in the preceding regression testing, the fitness function, f , is defined as follows:

$$f = \frac{\sum_{i=1}^m \left(hf_i \times \left(\sum_{j=TF_i}^n ht_j - (1/2)ht_{TF_i} \right) \right)}{\sum_{j=1}^n ht_j \times \sum_{i=1}^m hf_i} \quad (2)$$

In Eq. (2), hf_i is the fault severity of fault i in the preceding regression testing, and ht_j is the cost of j -th test case in the chromosome. TF_i is the TF_i -th test case in the chromosome that detects the fault i first. For example, let the information in Table 1 list the execution results of the test cases in the preceding regression testing. Table 2 shows the cost of test cases in Table 1. Table 3 lists the fault

Table 3

Example of exposed faults and severities.

Fault	Severity
1	2
2	1
3	4
4	5
5	2

Algorithm crossover**Input:**

parent₁: One selected chromosome in the population
parent₂: Another selected chromosome in the population
cp: Crossover probability

Output:

child₁, *child₂*: Two new chromosomes produced by the algorithm

```

1.  $r \leftarrow \text{genRandNum}(100)$ 
2. if  $r < cp$ 
3.    $p_1 \leftarrow \text{selectCrossPoint}(parent_1)$ 
4.    $p_2 \leftarrow \text{selectCrossPoint}(parent_2)$ 
5.    $firstSegment_1 \leftarrow \text{fragment}(p_1, parent_1)$ 
6.    $firstSegment_2 \leftarrow \text{fragment}(p_2, parent_2)$ 
7.    $child_1 \leftarrow \text{combine}(firstSegment_1, parent_2)$ 
8.    $child_2 \leftarrow \text{combine}(firstSegment_2, parent_1)$ 
9. else
10.   $child_1 \leftarrow parent_1$ 
11.   $child_2 \leftarrow parent_2$ 
12. return  $child_1, child_2$ 

```

Fig. 6. The crossover algorithm.

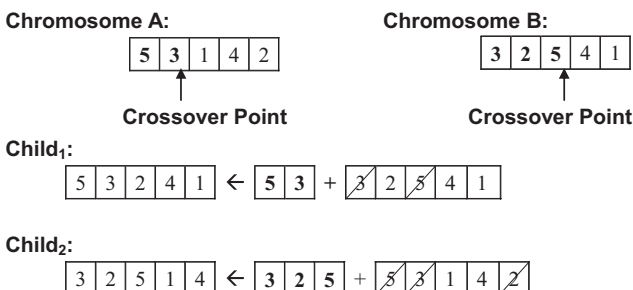
severities of the faults detected by the test cases. In this case, the fitness value of chromosome A is

$$\text{Fitness}_A = \frac{2 \left(\sum_{j=1}^5 ht_j - (2/2) \right) + 1 \left(\sum_{j=2}^5 ht_j - (1/2) \right) + 4 \left(\sum_{j=1}^5 ht_j - (2/2) \right) + 5 \left(\sum_{j=3}^5 ht_j - (4/2) \right) + 2 \left(\sum_{j=1}^5 ht_j - (2/2) \right)}{(4 + 2 + 1 + 3 + 2) \times (2 + 1 + 4 + 5 + 2)} = 0.789$$

3.2.2. Crossover

Crossover is typically a recombination process that combines the segments of one chromosome with the segments of another. The new chromosomes produced by crossover inherit some characteristics from both parents. The algorithm of the crossover operator is given in Fig. 6. The algorithm is the single point crossover. First, a random number, r , which ranges from 0 to 100, is generated on line 1. If r is less than the crossover probability, cp , the recombination process will begin on line 3. Otherwise, the child is the duplicate of the parent. When crossover is applied, the algorithm selects crossover points, p_1 and p_2 , for $parent_1$ and $parent_2$, respectively, on lines 3 and 4. On lines 5 and 6, the subsequences before the crossover point are then copied from both parents. The combined function on lines 7 and 8 generates a child by combining the copied subsequence of one parent with the genes of another parent that are not in the copied subsequence.

For example, considering the chromosomes in Fig. 3, Fig. 7 shows their crossover process. The crossover points of A and B

**Fig. 7.** Example of crossover.**Algorithm mutation****Input:**

child_c: Chromosome produced by crossover
mp: Mutation probability

Output:

child_m: Chromosome produced by the algorithm

```

1.  $r \leftarrow \text{genRandNum}(100)$ 
2. if  $r \leq mp$ 
3.    $p_1, p_2 \leftarrow \text{selectMutationPoint}()$ 
4.    $child_m \leftarrow \text{switchPosition}(p_1, p_2, child_c)$ 
5. else
6.    $child_m \leftarrow child_c$ 
7. return  $child_m$ 

```

Fig. 8. The mutation algorithm.

are at positions 2 and 3, respectively. Child₁ inherits the subsequence before the crossover point from A, and the rest from B. Because 3 and 5, which are genes of B, are also in the subsequence copied from A, they are not added to the child₁. Similarly, child₂ inherits the subsequence before the crossover point from B, and the genes that are not in that subsequence from A.

3.2.3. Mutation

The mutation operator on lines 10 and 11 in Fig. 2 is used to retain the diversity of chromosomes in the new population. If the chromosomes in the population are similar, the final solution achieved by the GA may fall into the local optimum solution. The chromosome produced by crossover is changed randomly by mutation to avoid becoming too similar to other chromosomes (Holland, 1992; Husbands, 1994). The algorithm of $\text{mutation}(child_c, mp)$ is shown in Fig. 8.

First, the $\text{mutation}(child_c, mp)$ generates a number, r , which ranges from 0 to 100 on line 1. If r is less than the mutation probability, mp , the algorithm selects two genes of $child_c$ randomly and switches their positions, as shown in Fig. 9. Otherwise, the mutation operator would not be applied. For the test case prioritization based on historical information, some test cases that had less fault detection effectiveness in the previous regression testing may have better fault detection effectiveness in future regression testing. The $\text{mutation}(child_c, mp)$ also gives those test cases a chance to get a higher priority for prioritization.

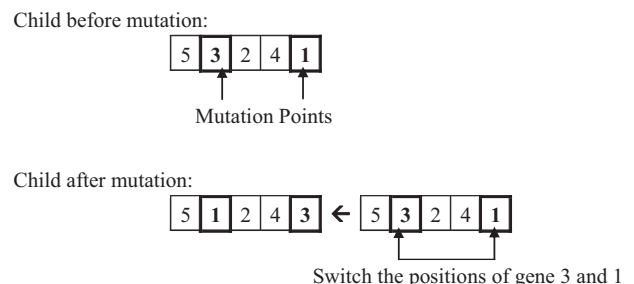
**Fig. 9.** Example of mutation.

Table 4
Characteristics of selected program – flex.

Version	Lines of code	No. of functions	No. of faults	No. of test cases
1	12,424	149	16	525
2	14,120	166	13	525
3	14,161	166	9	525
4	14,244	167	11	525
5	14,230	167	5	525

4. Experimental comparisons and evaluations

4.1. Experiment setup

In the past, Rothermel et al. (2001) distinguished two kinds of test case prioritizations including general test case prioritization and version-specific test case prioritization. The general test case prioritization prioritizes test cases for all modified versions of a program. In contrast, the version-specific test case prioritization prioritizes test cases for a specific modified version of a program. In this paper, we will focus on the method of version-specific test case prioritization. We consider two open-source tested programs obtained from the Software-artifact Infrastructure Repository (SIR). Note that all test case prioritization techniques used in the experiment are implemented by C language.

4.1.1. Selected programs

The characteristics of the selected programs used in the experiment are listed in Tables 4 and 5. Note that *flex* and *sed* are UNIX utility programs. *Flex* is used to recognize lexical patterns in the text, and *sed* is a stream editor which parses the input text, applies some operations, and then outputs the modified text. Here we plan to use five sequential versions and one test suite for each tested program from the Software-artifact Infrastructure Repository (URL: <http://sir.unl.edu>). The faults of each version are seeded by hand. Because neither the cost of each test case nor the severity of each fault is contained in the selected programs, we created these parts according to the function criticality (Malishevsky et al., 2006).

4.1.2. Selected prioritization techniques in comparison

In order to evaluate the performance of our proposed technique, in this paper some other test case prioritization techniques are also used for comparison. The information about these techniques is listed in Table 6. Note that in Table 6, the random and optimal techniques served as experimental controls. The GA-hist, hist-value and hist-fault-detected are history-based techniques which prioritize the test cases according to their historical records. The cc-total-fun, total-fun, and GA-fun are coverage-based techniques that schedule test cases based on their function coverage. Further, the GA-hist, hist-value, and cc-total-fun techniques are also developed for cost-cognizant test case prioritization. It is noticed that GA-hist is our proposed technique. GA-fun is the time-aware test case prioritization technique and Hist-value is the cost-cognizant test case prioritization technique. Some descriptions about these techniques are given as follows:

Table 5
I Characteristics of selected program – sed.

Version	Lines of code	No. of functions	No. of faults	No. of test cases
1	11,990	178	5	360
2	10,055	120	6	360
3	6671	259	4	360
4	6704	278	6	360
5	7164	285	4	360

Table 6
Information about test case prioritization techniques.

Abbreviation	Descriptions
Random	Order test case randomly
Optimal	The optimal solution for test case prioritization problem
hist-fault-detected	Prioritize historical fault detected effectiveness
GA-hist	Prioritize by genetic algorithm using historical records
Hist-value	Prioritize historical value considering cost and severity
GA-fun	Prioritize by genetic algorithm using coverage of functions
Total-fun	Prioritize coverage of functions
cc-total-fun	Prioritize coverage of functions considering cost and severity

- **Random prioritization (random):** Random prioritization randomly orders the test cases in the existing test suite.
- **Optimal prioritization (optimal):** Optimal prioritization seeks out the test order with the most effectiveness in terms of regression testing. We obtain such an order using the genetic algorithm and the known information, such as the execution result of the test cases and the existence of faults in regression testing. It is not a practicable technique because we cannot get known information before the regression testing. However, it provides a degree of intuition regarding how optimum the proposed technique is (Rothermel et al., 2001; Elbaum et al., 2002).
- **Historical fault detection effectiveness prioritization (hist-fault-detected):** This technique prioritizes test cases based on the historical value evaluated according to the fault detection effectiveness (Kim and Porter, 2002). The historical value of a test case is calculated as follows:

$$h_0 = f_1$$

$$h_k = \alpha f_k + (1 - \alpha)h_{k-1}, \quad 0 \leq \alpha \leq 1, \quad k \geq 1 \quad (3)$$

where h_k is the historical value of the test case in the k -th regression testing, f_k is the number of faults exposed by the test case in the k -th regression testing, and α is a smooth constant. When prioritizing test cases for the k -th regression testing, this technique calculates the historical value, h_{k-1} , for each test case and then sorts them in a decreasing order. Because we want f_k to dominate h_k , α is set as a higher value: 0.8.

- **Time-aware test suite prioritization (GA-fun):** Time-aware test suite prioritization uses genetic algorithm to reorder and select test cases in the constraint of total testing time. Its fitness function considers both the execution history and coverage and is calculated as follows:

$$fit(P, \sigma_j, tc, w) = [cc(P, \sigma_j, tc) \times w]$$

$$+ \frac{\sum_{i=1}^{|\sigma_j|} \text{execution} - \text{time}(\langle T_i \rangle) \times cc(P, \sigma_{j(i,j)}, tc)}{cc(P, \sigma_j, tc) \times \sum_{i=1}^{|\sigma_j|} \text{execution} - \text{time}(\langle T_i \rangle)}, \quad (4)$$

where cc is code coverage of the test suite (Walcott et al., 2006).

- **Total function coverage prioritization (total-fun):** Total function coverage prioritization orders test cases according to their function coverage. If the test case executes more functions, it will have a higher priority (Rothermel et al., 2001).
- **Cost-cognizant total function coverage prioritization (cc-total-fun):** This technique is a cost-cognizant test case prioritization technique adapted from the total function coverage prioritization (Malishevsky et al., 2006). When prioritization for the k -th regression testing is needed, this technique calculates

the award value of each test case. The award value of a test case is calculated as follows:

$$a_k = f_k \times \frac{\text{criticality}_{k-1}}{\text{cost}_{k-1}}, \quad (5)$$

where f_k is the number of functions executed by the test case in the k -th regression testing, criticality_{k-1} and cost_{k-1} are the total severity of the faults exposed by the test case and cost of the test case in the $k-1$ -th regression testing, respectively. The award value, a_k , is the product of f_k and the result of dividing criticality_{k-1} by cost_{k-1} . The test case with the greater award value is executed earlier.

4.1.3. Test-case cost and fault severity

The test case cost reflects the resources required when executing. There are various criteria used to measure the test case cost. For instance, the test case cost can be measured in terms of the actual time needed to execute it such as the machine time, human time, and test case execution time. It can also be measured by considering the monetary cost consumed by the test case. Similarly, the fault severity is related to the expense of correcting the fault, the loss of the company or user, and so forth. Therefore, software engineering can measure the fault severity considering the debugging time, the amount of money lost by the company, and so on (Malishevsky et al., 2006; Kaner et al., 2002; Black, 2004).

In our experiments, we try to measure the test case cost in terms of test case execution time. It is a common criterion used in most researches on test case prioritization (Malishevsky et al., 2006; Park et al., 2008; Elbaum et al., 2001). Further, we assign the fault severity according to the function (module) criticality presented in Malishevsky et al. (2006) and Elbaum et al. (2001). This measurement assumes that faults exposed in a function that has higher importance to the system are more severe than faults exposed in a less important function.

First, we randomly assign the importance value, which is an integer ranging from 1 to 5, to each function. This value reflects the importance of the function. If the function has a higher value, it is more important to the system. We then map the importance value of the function to the severity of faults exposed in it. Because the historical information is not available when prioritizing test cases for the first version of the tested programs, we manufacture it for the prioritization technique using such information. In this situation, the cost of each test case is 1 and the total detected fault severity of each test case is 0.

4.2. Effectiveness comparisons and discussion

Because random, GA-hist, and GA-fun could generate different test case orderings for regression testing, 1000 prioritizations were created by each of the methods for each regression testing. The average results of these prioritizations were then calculated. We compare the effectiveness of these selected techniques in Table 6 based on two conditions. In Condition 1, the cost and total detected fault severity of the test cases are various. In Condition 2, the cost and total detected fault severity of the test cases are uniform. We set each of them as 1. Since the APFDc value is a ratio, we present it in terms of a percentage. Furthermore, the parameters used by GA-fun and GA-hist are shown in Tables 7 and 8.

Condition 1: Variety test case cost and fault severity. In this condition, we consider a variety of test case costs and fault severities. Since the historical information is not available when ordering test cases for the first regression testing of the tested programs, thus GA-hist will degrade to a near random technique. Other prioritization techniques that utilize historical information, including hist-fault-detect, hist-value and cc-total-fun, will also degrade to random prioritization because the priority of all test cases

Table 7
Parameters used by GA-hist.

Parameter	Value
p (population size)	200
g (generation)	400
cp (crossover probability)	1
mp (mutation probability)	0.7
tc (test adequacy criteria)	Function
w (program coverage weight)	100

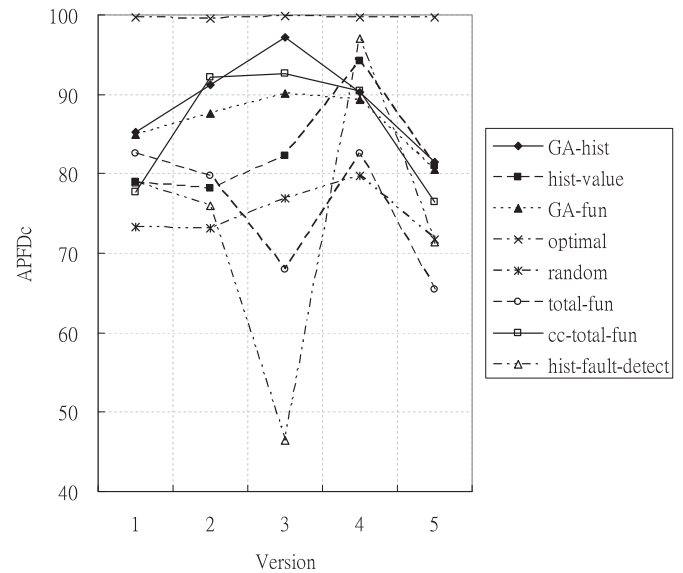


Fig. 10. The change of APFDc values through sed versions.

calculated by these techniques will be uniform. Tables 9 and 10 show the APFDc value of each technique from the experimental results of each version of sed and flex, respectively. The average APFDc of all versions is also shown in Tables 9 and 10. Further, the changes of APFDc values through sed versions and flex versions are plotted in Figs. 10 and 11.

Let's consider the mean APFDc values of the prioritization techniques in the experiment. As seen from Tables 9 and 10, GA-hist has a greater mean APFDc value than those of other techniques, except for the optimal technique on both sed and flex. Comparing GA-hist with random, GA-hist has a significant improvement on mean APFDc (improvement of 14.14% on sed and 11.68% on flex). GA-hist also performs well in comparison with history-based prioritization techniques such as hist-fault-detect and hist-value: improvements reached 6.16% and 6.23% on sed and flex (the maximum is 15.09%), respectively. In contrast with coverage-based prioritization techniques such as GA-fun, total-fun, and cc-total-fun, on average, GA-hist improves 2–4% on sed and flex, but is a significant improvement in comparison with total-fun on sed (improvement of 13.38%). Further, on sed, cc-total-fun, which is a cost-cognizant technique, has a higher performance than total-fun, which does not consider test case costs and fault severities

Table 8
Parameters used by GA-fun.

Parameter	Value
p (population size)	200
g (generation)	400
cp (crossover probability)	1
mp (mutation probability)	0.7
tc (test adequacy criteria)	Function
w (program coverage weight)	100

Table 9

APFDc values of the prioritization techniques for tested program – sed.

Version	1	2	3	4	5	Mean	Improvement
Random	73.26%	73.09%	76.89%	79.69%	71.69%	74.92%	+14.14%
Optimal	99.63%	99.59%	99.78%	99.68%	99.66%	99.67%	–10.61%
hist-fault-detect	78.92%	75.94%	46.51%	97.03%	71.44%	73.97%	+15.09%
GA-hist	85.26%	91.17%	97.11%	90.28%	81.49%	89.06%	–
hist-value	78.92%	78.24%	82.29%	94.12%	80.92%	82.90%	+6.16%
GA-fun	84.87%	87.66%	90.17%	89.27%	80.57%	86.51%	+2.55%
Total-fun	82.62%	79.75%	67.93%	82.63%	65.45%	75.68%	+13.38%
cc-total-fun	77.64%	92.16%	92.59%	90.35%	76.41%	85.83%	+3.23%

Table 10

APFDc values of the prioritization techniques for tested program – flex.

Version	1	2	3	4	5	Mean	Improvement
Random	91.50%	73.63%	57.87%	88.33%	99.43%	82.15%	+11.68%
Optimal	99.79%	99.47%	99.39%	99.87%	99.91%	99.69%	–5.86%
hist-fault-detect	93.05%	84.60%	70.21%	90.21%	99.91%	87.60%	+6.23%
GA-hist	98.94%	87.13%	84.17%	98.98%	99.91%	93.83%	–
hist-value	93.05%	77.71%	73.87%	88.10%	99.91%	86.53%	+7.30%
GA-fun	98.47%	86.78%	70.39%	98.83%	99.91%	90.88%	+2.95%
Total-fun	96.44%	83.11%	71.52%	99.30%	99.51%	89.98%	+3.85%
cc-total-fun	91.56%	84.85%	72.65%	99.55%	99.91%	89.65%	+4.18%

when prioritizing. On flex, the relationship is the reverse: total-fun is better than cc-total-fun, but the difference is not significant.

Next, let's consider the experimental results of each version presented in Figs. 10 and 11. It can be seen that GA-hist provides high performance for the first regression testing on both sed and flex while it degrades to near random prioritization, as described earlier. On flex, GA-hist usually has a higher APFDc value except in version 4. Total-fun and cc-total-fun perform better than GA-hist in this version, but the difference is insignificant. On sed, GA-hist also improves the APFDc value on average. In version 2, although GA-hist shows a lower APFDc value in comparison with cc-total, it is also insignificant.

From Fig. 10, we find that some techniques such as hist-value and hist-fault-detect have greater APFDc values than GA-hist in version 4. This results from the fault exposed by a few test cases whose historical records are worse or the same as other test cases in this version. For instance, the fault 6 is detected by test cases 34, 321, and 339 in version 4 of the experiment. According to the historical information, we find that test cases 321 and 339 did not detect

any faults in version 3. Moreover, there are 17 test cases that have the same fault detection capability as test case 34. Because hist-value and hist-fault-detect give the same rank to test cases that have the same capability, test case 34 and those 17 test cases will be executed earlier, while test cases 321 and 339 will be executed later. However, since GA-hist randomly orders the test cases that have the same ability, test cases 34, 321 and 339 may get higher or lower priority individually. If they all get lower priority, fault 6 will be exposed later, and such an order will obtain a lower APFDc value. Therefore, the average APFDc value of 1000 prioritizations created by GA-hist is lower than the average APFDc value of hist-value and hist-fault-detect. There is a common assumption for almost all history-based techniques that the fault discovery capability of each test case does not vary a lot in the next version (Park et al., 2008), and this assumption holds in general cases (Kim and Porter, 2002).

Condition 2: Uniform test case cost and fault severity. In this condition, we consider uniform test case cost and fault severity. As described previously, the formula for APFDc will be reduced to the formula for APFD. In other words, the APFDc metric produced in this condition is equivalent to the APFD metric. The experimental results are shown in Tables 11 and 12. Figs. 12 and 13 also show the changes of APFDc values through the sed versions and flex versions, respectively. First, we consider the mean APFDc value of the prioritization techniques. As seen from Tables 11 and 12, GA-hist provides higher effectiveness compared with other techniques, other than the optimal technique on both sed and flex. As in condition 1, GA-hist is a significant improvement in comparison with random (improvement of at least 10% on APFDc). Compared with history-based prioritization techniques, GA-hist improves approximately 10% on sed and 5% on flex. In contrast to coverage-based prioritization techniques except GA-fun, GA-hist improves 3–8% on sed and flex. The effectiveness of GA-hist is almost equal to the effectiveness of GA-fun on both tested programs.

Next, considering Figs. 12 and 13, we find that the changes in condition 2 are very similar to the changes in condition 1. On sed, GA-hist also obtains a lower APFDc value than cc-total and hist-fault-detect in versions 2 and 3, respectively, and gets the highest APFDc value in the other versions. On flex, however, it is surprising that cc-total performs better than the other techniques except for the optimal technique in version 2, but shows the worst effectiveness in version 3. According to Eq. (3), cc-total-fun schedules

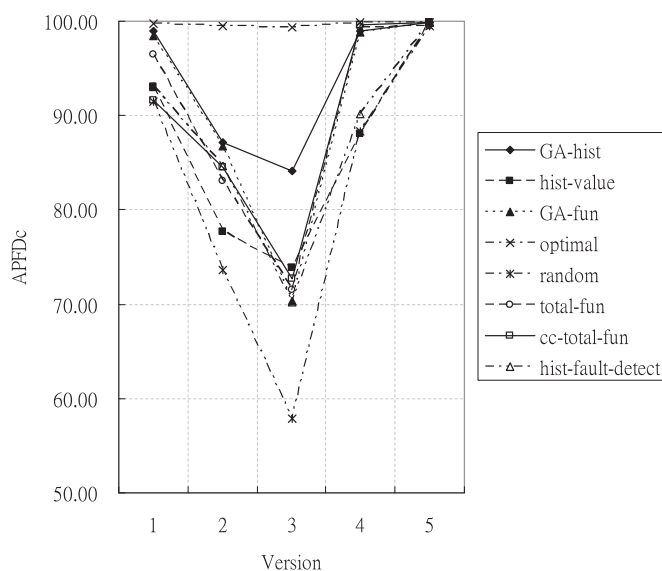
**Fig. 11.** The change of APFDc values through flex versions.

Table 11
APFDc values of the prioritization techniques for tested program – sed.

Version	1	2	3	4	5	Mean	Improvement
Random	76.99%	80.38%	64.55%	74.88%	79.91%	75.34%	+16.00%
Optimal	99.67%	99.47%	99.63%	99.81%	99.61%	99.64%	–8.30%
hist-fault-detect	85.03%	82.50%	55.97%	97.36%	78.13%	79.80%	+11.54%
GA-hist	89.39%	91.16%	96.49%	94.23%	85.43%	91.34%	–
hist-value	85.02%	82.50%	71.80%	94.40%	73.61%	81.47%	+9.87%
GA-fun	88.60%	90.66%	91.46%	93.63%	82.86%	89.44%	+1.09%
Total-fun	88.81%	86.90%	74.51%	92.22%	72.50%	82.99%	+8.35%
cc-total-fun	82.69%	92.87%	86.53%	96.75%	78.40%	87.45%	+3.89%

Table 12
APFDc values of the prioritization techniques for tested program – flex.

Version	1	2	3	4	5	Mean	Improvement
Random	94.18%	80.87%	60.70%	97.42%	96.62%	85.96%	+9.10%
Optimal	99.82%	99.63%	99.44%	99.87%	99.90%	99.73%	–4.67%
hist-fault-detect	95.48%	90.95%	72.70%	92.18%	99.90%	90.24%	+4.82%
GA-hist	99.29%	92.48%	84.43%	99.18%	99.90%	95.06%	–
hist-value	95.48%	90.95%	69.55%	90.95%	99.90%	89.37%	+5.69%
GA-fun	99.27%	91.93%	81.68%	99.03%	99.90%	94.36%	+0.70%
Total-fun	97.78%	89.78%	73.59%	99.38%	99.56%	92.02%	+3.04%
cc-total-fun	95.77%	96.63%	64.77%	99.52%	99.90%	91.32%	+3.07%

test cases based on its function coverage and its historical information. Compared with total-fun, which just considers the function coverage, we find that the prioritization technique that considers historical information can improve the effectiveness of regression testing, except in version 3. Further, the effectiveness of history-based techniques such as hist-value and hist-fault-detect is lower than the coverage-based techniques in version 3. We also find that even though the historical information may be useless in this version, GA-hist can provide better performance than the other techniques.

5. Impact of generations and population size

5.1. Efficiency comparison and analysis

In this section, we will compare and discuss the efficiency of GA-hist with the efficiency of GA-fun; both of them are GA-based prioritization techniques. Efficiency is measured in terms of the execution time. We compare their efficiency with various

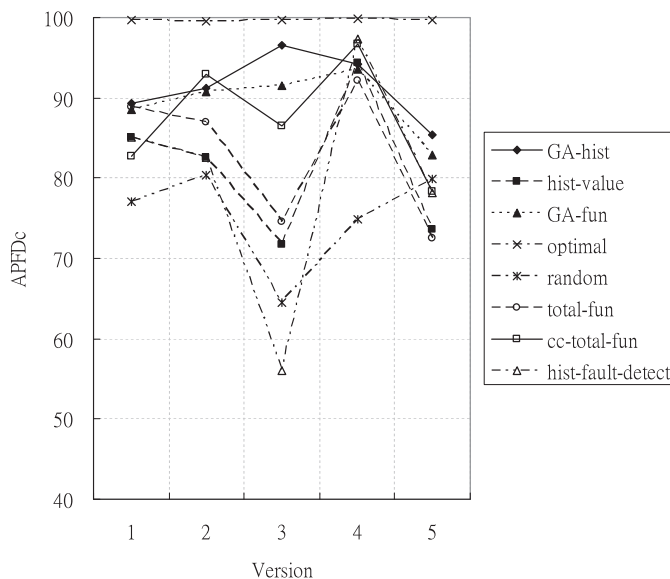


Fig. 12. The change trends of APFDc values through sed versions.

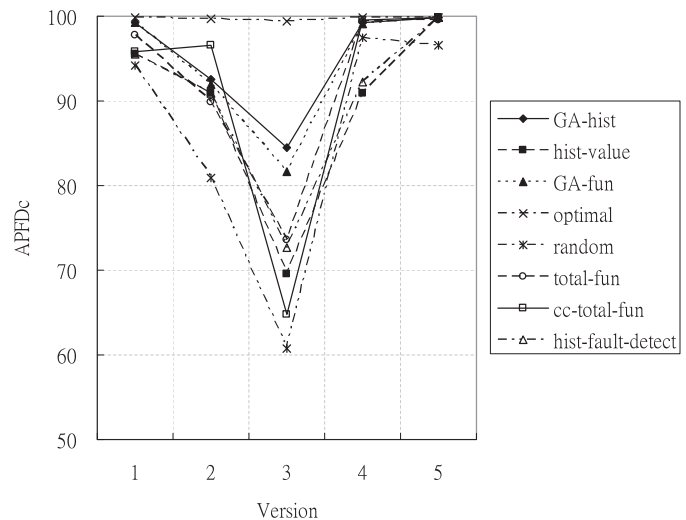


Fig. 13. The change trends of APFDc values through flex versions.

parameters such as the number of generation and the population size. Some experimental results are shown in Table 13. The second column indicates the various combinations of number of generation and the population size. The third and the fourth columns show the average execution time, calculated by averaging the execution time of each version.

As seen from Table 13, we find that (a) when the product of population size and the number of generations is smaller, the average

Table 13
The average execution time of GA-hist and GA-fun.

Program	# of generations, population size	Average execution time (s)	
		GA-hist	GA-fun
Sed	(100, 400)	25.56	154.22
	(250, 250)	42.86	317.82
	(400, 100)	27.26	204.02
Flex	(100, 400)	57.65	440.48
	(250, 250)	97.22	875.40
	(400, 100)	64.51	558.57

Table 14

Impact of number of generations and population size on APFDc values.

# of generation, population size	Sed version					Average APFDc
	1	2	3	4	5	
(1000, 100)	86.31%	93.27%	97.28%	89.88%	82.13%	89.77%
(500, 100)	83.86%	91.45%	96.99%	90.24%	82.15%	88.94%
(400, 100)	85.26%	91.17%	97.11%	90.28%	81.49%	89.06%
(300, 100)	85.28%	90.66%	97.13%	91.03%	82.08%	89.24%
(200, 100)	84.46%	90.17%	96.83%	91.75%	80.78%	88.80%
(100, 100)	85.47%	88.16%	95.72%	93.67%	78.35%	88.27%
(50, 100)	83.86%	88.01%	95.71%	93.92%	78.82%	88.06%
(100, 50)	85.70%	83.93%	94.73%	93.53%	75.53%	86.68%
(100, 200)	85.98%	88.04%	95.84%	93.56%	78.04%	88.29%
(100, 300)	84.44%	87.47%	95.63%	93.50%	78.44%	87.90%
(100, 400)	85.70%	87.64%	95.69%	93.62%	77.58%	88.05%
(100, 500)	84.83%	87.79%	95.65%	94.09%	77.77%	88.03%
(100, 1000)	85.17%	87.73%	95.45%	93.75%	76.99%	87.82%

Table 15

Impact of number of generations and population size on APFDc values.

# of generation, population size	Flex version					Average APFDc
	1	2	3	4	5	
(1000, 100)	98.93%	87.29%	83.77%	99.04%	99.91%	93.79%
(500, 100)	99.07%	87.53%	84.79%	99.01%	99.91%	94.06%
(400, 100)	98.94%	87.13%	84.17%	98.98%	99.91%	93.83%
(300, 100)	98.70%	86.56%	83.08%	98.99%	99.91%	93.45%
(200, 100)	99.13%	86.60%	82.70%	98.91%	99.91%	93.45%
(100, 100)	98.98%	86.19%	81.64%	98.80%	99.91%	93.10%
(50, 100)	99.19%	83.12%	80.09%	98.84%	99.91%	92.23%
(100, 50)	98.83%	85.13%	80.77%	98.96%	99.91%	92.72%
(100, 200)	98.92%	84.78%	81.39%	98.79%	99.91%	92.76%
(100, 300)	98.77%	84.47%	81.08%	98.68%	99.91%	92.58%
(100, 400)	98.97%	84.04%	82.69%	98.88%	99.91%	92.90%
(100, 500)	98.81%	84.64%	80.37%	98.86%	99.91%	92.52%
(100, 1000)	98.96%	83.96%	80.76%	98.69%	99.91%	92.46%

execution time is lower, and (b) the results indicate that GA-hist is much more efficient than GA-fun. This results from the time overhead consumed by calculating the fitness of each chromosome. Fitness evaluation is time-intensive (Walcott et al., 2006) and the number of times to evaluate the fitness is proportional to the product of population size and the number of generations. Further, the time for generating the first population also affects the efficiency. Therefore, although both the (100, 400) configuration and (400, 100) configuration should calculate fitness 40,000 times, the (400, 100) configuration has a lower execution time.

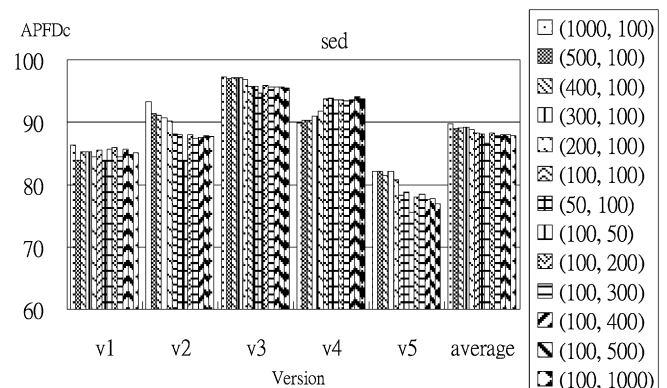
5.2. Effectiveness analysis

As described previously, the number of generations and population size affect the execution time of the proposed technique. In this section, we discuss the influence of the number of generations and population size on APFDc values. Tables 14 and 15 show the APFDc values on sed and flex, respectively, when various numbers of generations and population sizes are applied. The seventh columns of Tables 14 and 15 present the average APFDc values of all versions. The change trends of the APFDc values are revealed in Figs. 14 and 15.

First, we consider the APFDc value of each version. As seen from Figs. 14 and 15, when the number of generations is greater than the population size, the proposed technique provides a higher APFDc value. However, the relationship is reversed for version 4 of sed. The number of generations implies the number of times the crossover operation is executed. When the crossover is performed more times, the test cases with poorer historical records in terms of their potential ability to detect faults in the future have a greater chance to get a higher priority. So the proposed technique leads to

a higher APFDc if the number of generations is greater. In version 4 of sed, we find the faults exposed by a few test cases as described in the preceding section. In this situation, applying the crossover more times may have a negative influence because these test cases have a greater chance to get a lower priority. Therefore, the proposed technique provides a lower APFDc for this version when the number of generations is greater.

The population size also affects the effectiveness of the proposed technique on sed. If the size is too small, the proposed technique will have a low APFDc value. Finally, we consider the average APFDc values when various numbers of generations and population sizes are applied. As seen from Figs. 14 and 15, the results also show that the proposed technique has better effectiveness when the number of generations is greater, but the difference is not significant.

**Fig. 14.** The trend of APFDc values on sed.

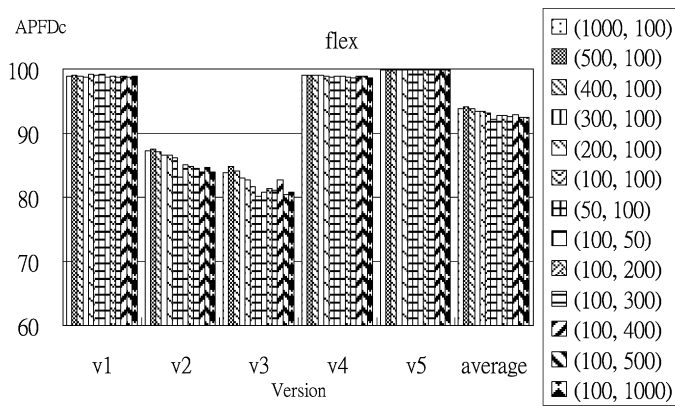


Fig. 15. The trend of APFDc values on flex.

6. Conclusions

Most test case prioritization techniques aim to schedule test cases in such a way as to increase the rate of fault detection for (security) regression testing. They assume that all test case costs and fault severities are equal, and they prioritize test cases according to information acquired by analyzing the source code. In fact, test case costs and fault severities usually vary, and testers may not obtain the source code before prioritization. In this paper, we propose a cost-cognizant prioritization technique that orders test cases according to their historical information. The proposed technique prioritizes test cases according to their various test costs and fault severities without analyzing the source code. Compared with other traditional history-based prioritization techniques, through mutation, the proposed technique gives the test cases that have poor historical records a greater chance to get a higher priority. If some of these test cases can have a better ability to detect faults in the next regression testing, the performance of the proposed technique will not decrease. Further, the proposed technique also avoids situations where the test cases with equivalent ability in the previous regression testing are given the same rank, and it can also improve the performance of prioritization.

In our experiments, we have used two UNIX utility programs, including sed and flex, to evaluate the proposed technique. The results have shown that on average, the proposed technique can significantly improve the effectiveness in comparison with three coverage-based techniques and two other history-based techniques. It also performed well when test case costs and fault severities were equal. In contrast to the other GA-produced prioritization technique used in the experiment, the proposed technique provided higher efficiency because the number of faults was far fewer than the number of functions. According to the experimental results, we verify that the proposed technique is useful. We also confirm that prioritizing test cases based on their historical information can provide high effectiveness. Finally, there are still some research issues such as improving the effectiveness of (security) regression testing by incorporating other historical information, combining history-based techniques with coverage-based techniques, severity models for security regression testing, resource-constrained security regression testing, etc. We are currently working on some of these issues and plan to publish the research results in the future.

Acknowledgments

The work described in this paper was supported by the National Science Council, Taiwan, under Grants NSC 98-2221-E-007-067, NSC 99-2815-C-007-044-E, and NSC 100-2220-E-007-006. The

authors would like to express their gratitude to the associate editor and anonymous reviewers for their insightful comments, encouragements, and suggestions.

References

- Ammann, P., Offutt, J., 2008. Introduction to Software Testing. Cambridge University Press.
- Binkley, D., 1997. Semantics guided regression test cost reduction. *IEEE Transactions on Software Engineering* 23 (8), 498–516.
- Black, R., 2004. Critical Testing Processes. Addison-Wesley, Boston, MA, USA.
- Boehm, B., Huang, L., 2003. Value-based software engineering: a case study. *IEEE Computer* 36 (3), 33–41.
- Chen, T.Y., Lau, M.F., 1998. A simulation study on some heuristics for test suite reduction. *Information and Software Technology* 40 (13), 777–787.
- Do, H., Rothermel, G., Kinneer, A., 2004. Empirical studies of test case prioritization in a Junit testing environment. In: *Proceedings of the 23rd International Symposium on Software Reliability Engineering (ISSRE'04)*, St. Malo, France, pp. 113–124.
- Elbaum, S., Malishevsky, A.G., Rothermel, G., 2002. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering* 28 (2), 159–182.
- Elbaum, S., Malishevsky, A.G., Rothermel, G., 2001. Incorporating varying test costs and fault severities into test case prioritization. In: *Proceedings of the 23rd International Conference on Software Engineering (ICSE 2001)*, Toronto, Canada, pp. 329–338.
- Elbaum, S., Rothermel, G., Kanduri, S., Malishevsky, A.G., 2004. Selecting a cost-effective test case prioritization technique. *Software Quality Control* 12 (3), 185–210.
- Engstrom, E., Runeson, P., Skoglund, M., 2010. A systematic review on regression test selection techniques. *Information and Software Technology* 52 (1), 14–30.
- Goel, A.L., Okumoto, K., 1978. An analysis of recurrent software errors in a real-time control system. In: *Proceedings of the 1978 Annual Conference (ACM'78)*, Washington, DC, pp. 496–501.
- Graves, T.L., Harrold, M.J., Kim, J.M., Porter, A., Rothermel, G., 2001. An empirical study of regression test selection techniques. *ACM Transactions on Software Engineering and Methodology* 10 (2), 184–208.
- Harrold, M.J., Gupta, R., Soffa, M.L., 1993. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology* 2 (3), 270–285.
- Huang, C.Y., 2005. Performance analysis of software reliability growth models with testing-effort and change-point. *Journal of Systems and Software* 76 (2), 181–194.
- Husbands, P., 1994. Genetic Algorithms for Scheduling. *AISB Quarterly*, No. 89.
- Jeffrey, D., Gupta, N., 2006. Test case prioritization using relevant slices. In: *Proceedings of the 30th IEEE Annual International Computer Software and Applications Conference (COMPSAC 2006)*, Chicago, Illinois, USA, pp. 411–420.
- Holland, J.H., 1992. Adaptation in Natural and Artificial Systems. MIT Press, Cambridge, MA, USA.
- Kaner, C., Bach, J., Pettichord, B., 2002. Lessons Learned in Software Testing. John Wiley & Sons, New York, USA.
- Karlsson, J., Ryan, K., 1997. A cost-value approach for prioritizing requirements. *IEEE Software* 14 (5), 67–74.
- Kim, J.M., Porter, A., 2002. A history-based test prioritization technique for regression testing in resource constrained environments. In: *Proceedings of the International Conference on Software Engineering (ICSE 2002)*, Orlando, FL, USA, pp. 119–129.
- Kim, J.M., Porter, A., Rothermel, G., 2002. An empirical study of regression test application frequency. In: *Proceedings of the 24th International Conference on Software Engineering (ICSE 2002)*, Orlando, FL, USA, pp. 126–135.
- Leung, H.K.N., White, L.J., 1990. A study of integration testing and software regression at the integration level. In: *Proceedings of the IEEE International Conference on Software Maintenance*, San Diego, pp. 290–300.
- Lin, C.T., Huang, C.Y., 2009a. Staffing level analysis of software debugging through rate-based simulation approaches. *IEEE Transactions on Reliability* 58 (4), 711–724.
- Lin, J.W., Huang, C.Y., 2009b. Analysis of test suite reduction with enhanced tie-breaking techniques. *Information and Software Technology* 51 (4), 679–690.
- Li, Z., Harman, M., Hierons, R.M., 2007. Search algorithms for regression test case prioritization. *IEEE Transactions on Software Engineering* 33 (4), 225–237.
- Lyu, M.R., 1996. Handbook of Software Reliability Engineering. McGraw-Hill, p. 851.
- Malishevsky, A.G., Ruthruff, J.R., Rothermel, G., Elbaum, S., 2006. Cost-cognizant Test Case Prioritization Technical Report TR-UNL-CSE-2006-0004. University of Nebraska-Lincoln.
- Mei, L., Zhang, Z., Chan, W.K., Tse, T.H., 2009. Test case prioritization for regression testing of service-oriented business applications. In: *Proceedings of the 18th ACM International World Wide Web Conference (WWW 2009)*, Madrid, Spain, pp. 901–910.
- Nunes, F.J.B., Belchior, A.D., Albuquerque, A.B., 2010. Security engineering approach to support software security. In: *Services (SERVICES-1)*, 2010 6th World Congress on, vol. no, pp. 48–55.
- Ohba, M., Chou, X., 1989. Does imperfect debugging affect software reliability growth. In: *Proceedings of the 11th International Conference on Software Engineering (ICSE'89)*, Pittsburgh, PA, USA, pp. 237–244.

- Park, H., Ryu, H., Baik, J., 2008. Historical value-based approach for cost-cognizant test case prioritization to improve the effectiveness of regression testing. In: *Proceedings of the 2nd International Conference on Secure System Integration and Reliability Improvement (SSIRI 2008)*, Yokohama, Japan, pp. 39–46.
- Qu, B., Nie, C., Xu, B., Zhang, X., 2007. Test case prioritization for black box testing. In: *Proceedings of the 31st IEEE Annual International Computer Software and Applications Conference (COMPSAC 2007)*, Beijing, China, pp. 465–474.
- Rothermel, G., Harrold, M.J., 1997. A safe, efficient regression test selection technique. *ACM Transactions on Software Engineering and Methodology* 6 (2), 173–210.
- Rothermel, G., Untch, R.H., Chu, C., Harrold, M.J., 2001. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering* 27 (10), 929–948.
- Sheriff, M., Lake, M., Williams, L., 2007. Prioritization of regression tests using singular value decomposition with empirical change records. In: *Proceedings of the 18th IEEE International Symposium on Software Reliability Engineering (ISSRE 2007)*, Trollhättan, Sweden, pp. 81–90.
- Sharma, V.S., Trivedi, K.S., 2007. Quantifying software performance, reliability and security: an architecture based approach. *Journal of Systems and Software* 80 (4), 493–509.
- Srikanth, H., Williams, L., Osborne, J., 2005. System test case prioritization of new and regression test cases. In: *Proceeding of the 4th International Symposium on Empirical Software Engineering (ISESE)*, Noosa Heads, Australia, p. 10.
- Walcott, K.R., Soffa, M.L., Kapfhammer, G.M., Roos, R.S., 2006. Time-aware test suite prioritization. In: *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2006)*, Portland, ME, USA, pp. 1–12.
- Zengkai, M., Jianjun, Z., 2008. Test case prioritization based on analysis of program structure. In: *Proceeding of 15th IEEE Asia-Pacific Software Engineering Conference (APSEC 2008)*, Beijing, China, pp. 471–478.
- Zhong, H., Zhang, L., Mei, H., 2008. An experimental study of four typical test suite reduction techniques. *Information and Software Technology* 50 (6), 534–546.
- Yu-Chi Huang** received his MS degree (2009) from the Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan. Currently, he is working at ASUSTeK Computer Inc., Taipei, Taiwan.
- Kuan-Li Peng** received his BS degree (2007) and MS degree (2009) in Computer Science from National Tsing Hua University, Hsinchu, Taiwan. He is currently pursuing his PhD degree in the Department of Computer Science at National Tsing Hua University. His research interests include software quality assurance and Web service quality assurance.
- Chin-Yu Huang** is currently an Associate Professor in the Department of Computer Science at National Tsing Hua University, Hsinchu, Taiwan. He received the M.S. (1994), and the Ph.D. (2000) in Electrical Engineering from National Taiwan University, Taipei. He was with the Bank of Taiwan from 1994 to 1999, and was a senior software engineer at Taiwan Semiconductor Manufacturing Company (TSMC) from 1999 to 2000. Before joining NTHU in 2003, he was a division chief of the Central Bank of China, Taipei. He received the Ta-You Wu Memorial Award from the National Science Council of Taiwan in 2008. In 2009, he was ranked as the top 15 scholars in Systems and Software Engineering worldwide between 2002 and 2006 by the *Journal of Systems and Software* based on his research on software reliability, software testing, and software metrics. He received an Honorable Mention Best Paper Award at the 2010 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM 2010). His research interests are software reliability engineering, software testing, software metrics, software testability, and fault tree analysis.